FCM 2: Assignment 2

Dr. Gallivan Spring 2026

Alex Sampson

Contents

1 Executive Summary **3**

2 Statement of the Problem **3**

3 Mathematical Description **4**
- 3.1 Polynomial Interpolation . . . 4
- 3.2 Barycentric Interpolation . . . 4
- 3.3 Newton Form . . . 4
- 3.4 Lebesgue Function and Conditioning . . . 5
- 3.5 Floating Point Error Model and Stability Bound . . . 5

4 Implemented Subroutines **6**
- 4.1 Mesh (Node) Generation . . . 6
- 4.2 Product Evaluation for Validation . . . 6
- 4.3 Barycentric Form 1 Diagnostics . . . 6
- 4.4 Barycentric Form 2 Evaluation . . . 6
- 4.5 Newton Form: Coefficients and Evaluation . . . 6
- 4.6 Lebesgue Constant and Conditioning Quantities . . . 7
- 4.7 Hilbert Matrix Diagnostic . . . 7

5 Driver Programs and Experiments **7**
- 5.1 Task 1: Code Validation . . . 7
- 5.2 Task 2: Interpolation of $f_1(x) = (x-2)^9$ . . . 7
- 5.3 Task 3: Interpolation of $f_2(x) = \prod_{i=1}^{d}(x-i)$ . . . 7
- 5.4 Task 4: Interpolation of $f_3(x) = \ell_n(x)$ . . . 7
- 5.5 Task 5: Interpolation of $f_4(x) = 1/(1+25x^2)$ . . . 8

6 Results and Discussion **8**
- 6.1 Task 1: Code Validation . . . 8
- 6.2 Task 2: Interpolation of $f_1(x) = (x-2)^9$ . . . 9
- 6.3 Task 3: Interpolation of $f_2(x) = \prod_{i=1}^{d}(x-i)$ . . . 9
- 6.4 Task 4: Interpolation of $f_3(x) = \ell_n(x)$ . . . 10
- 6.5 Task 5: Runge's Function and Convergence . . . 11

7 Conclusions **12**

8 Disclaimer **13**

A Appendix: Figures **14**

1 Executive Summary

This project investigates the numerical stability, conditioning, and convergence behavior of polynomial interpolation through a sequence of computational experiments. The central question is how the accuracy of the computed interpolation depends on three choices:

1. the mesh distribution (**uniform** vs. **Chebyshev**),
2. the evaluation representation (**barycentric Form 2** vs. **Newton form)**,
3. and the ordering of the nodes.

The experiments demonstrate that the growth of the Lebesgue constant Λ_n is the principal driver of error amplification, and that this growth is controlled almost entirely by the mesh distribution. **Uniform meshes** produce exponential growth in Λ_n that rapidly overwhelms any stability advantage of the evaluation form. **Chebyshev meshes** keep Λ_n nearly constant, enabling stable convergence to the precision floor of the arithmetic. Among evaluation forms, **barycentric Form 2** is the most reliably stable. **Newton interpolation** is highly sensitive to node ordering, but **Leja ordering** recovers barycentric level accuracy in nearly every case tested.

2 Statement of the Problem

Polynomial interpolation is a classical approximation tool: given $n+1$ distinct nodes and function values, there is a unique polynomial of degree at most n agreeing with those values. The approximation problem is well understood analytically, but the numerical behavior of the computation depends heavily on how the interpolation is evaluated and on the distribution of the interpolation nodes.

The project addresses the following questions:

1. How does the choice of **interpolation nodes** affect numerical stability and accuracy?
2. How do different **evaluation representations** influence rounding error propagation?
3. How well does the **Lebesgue constant** predict observed error growth?
4. How does **node ordering** affect the Newton form?
5. How do the theoretical predictions of Higham (2004) and Berrut & Trefethen (2004) compare against empirical results?

Experiments are conducted using **uniformly spaced** and **Chebyshev nodes**, with interpolants evaluated in **barycentric Form 2** and **Newton form** under several node orderings. The experiments track error and conditioning as polynomial degree increases.

3 Mathematical Description

3.1 Polynomial Interpolation

Let $x_0, x_1, \ldots, x_n$ be $n+1$ distinct nodes in $\mathbb{R}$ and let f be a function defined on an interval containing these nodes. The interpolation problem seeks a polynomial p_n of degree at most n satisfying

$$p_n(x_i) = f(x_i), \quad i = 0, 1, \ldots, n.$$

Existence and uniqueness are classical. One explicit representation is the **Lagrange form**

$$p_n(x) = \sum_{j=0}^{n} f(x_j)\,\ell_j(x), \qquad \ell_j(x) = \prod_{\substack{k=0 \\ k\neq j}}^{n} \frac{x - x_k}{x_j - x_k},$$

where $\{\ell_j\}$ are the Lagrange basis polynomials. This form is mathematically transparent but numerically unstable in direct evaluation.

3.2 Barycentric Interpolation

A numerically stable reformulation uses the **barycentric weights**

$$w_j = \frac{1}{\displaystyle\prod_{\substack{k=0 \\ k\neq j}}^{n} (x_j - x_k)},$$

through which the interpolant is expressed in **barycentric Form 2** as

$$p_n(x) = \frac{\displaystyle\sum_{j=0}^{n} \frac{w_j\, f(x_j)}{x - x_j}}{\displaystyle\sum_{j=0}^{n} \frac{w_j}{x - x_j}}.$$

This representation avoids constructing the Lagrange basis explicitly and is the standard recommendation for numerical evaluation of polynomial interpolation.

3.3 Newton Form

The **Newton form** writes the interpolation as

$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n \prod_{k=0}^{n-1} (x - x_k),$$

with coefficients a_k determined by divided differences. This form is convenient because new nodes can be incorporated without recomputing from scratch. However, its numerical behavior depends strongly on node ordering, and poor orderings can lead to significant rounding error growth.

3.4 Lebesgue Function and Conditioning

The conditioning of the interpolation problem is characterized by the Lebesgue function

$$\Lambda_n(x) = \sum_{j=0}^{n} |\ell_j(x)|$$

and the Lebesgue constant $\Lambda_n = \max_x \Lambda_n(x)$. The classical estimate

$$\|f - p_n\|_\infty \leq (1 + \Lambda_n)\, \|f - p_n^*\|_\infty, \qquad \text{(see Quarteroni property 8.1)}$$

where p_n^* is the best degree-n approximation to f, shows that Λ_n acts as a condition number for the interpolation problem. For **uniformly spaced nodes** Λ_n grows exponentially in n. For **Chebyshev nodes** it grows only logarithmically. This contrast is the root cause of most phenomena observed in the experiments below.

3.5 Floating Point Error Model and Stability Bound

All computations are performed in floating point arithmetic. The standard model assumes each basic arithmetic operation satisfies

$$fl(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \leq u,$$

where u is the unit roundoff ($u \approx 6 \times 10^{-8}$ in single precision, $u \approx 10^{-16}$ in double precision).

A detailed forward error analysis of **barycentric Form 2** is given in Higham (2004, Theorem 4.1 and Corollary 4.2). To state the result, define the pointwise conditioning quantities

$$\kappa(x, n, f) = \frac{\displaystyle\sum_{j=0}^{n} \frac{|w_j|\,|f(x_j)|}{|x - x_j|}}{\left|\displaystyle\sum_{j=0}^{n} \frac{w_j\, f(x_j)}{x - x_j}\right|}, \qquad \kappa(x, n, 1) = \frac{\displaystyle\sum_{j=0}^{n} \frac{|w_j|}{|x - x_j|}}{\left|\displaystyle\sum_{j=0}^{n} \frac{w_j}{x - x_j}\right|},$$

where $\{w_j\}$ are the **barycentric weights** and $\{x_j\}$ are the interpolation nodes. The quantity $\kappa(x, n, f)$ measures the sensitivity of the numerator of the barycentric formula to perturbations in the function values $f(x_j)$, while $\kappa(x, n, 1)$ measures the sensitivity of the denominator, which is independent of f. Note that $\mathrm{cond}(x, n, 1) = \Lambda_n(x)$, the Lebesgue function evaluated at x.

Higham (2004, Corollary 4.2) shows that the forward error in the computed interpolation $\widehat{p}_n(x)$ satisfies

$$|\widehat{p}_n(x) - p_n(x)| \leq \big[(3n + 4)\, u\, \kappa(x, n, f) + (3n + 2)\, u\, \kappa(x, n, 1)\big]\, |p_n(x)|,$$

to first order in u. This bound has two terms: the first reflects cancellation in the numerator of the **barycentric formula** and grows with the sensitivity of the function values, while the second reflects cancellation in the denominator and is governed entirely by the node distribution through $\Lambda_n(x)$. When the **Lebesgue constant** is large, the second term dominates regardless of f. This bound is used directly to produce the error envelopes in the experiment figures.

4 Implemented Subroutines

All routines were implemented in MATLAB. The following is a brief description of what each routine does; the task drivers in Section 5 call these routines to carry out the experiments.

4.1 Mesh (Node) Generation

The interpolation nodes $\{x_0, \ldots, x_n\}$ are generated on $[a, b]$ using three mesh routines. `mesh_uniform.m` produces equally spaced nodes $x_j = a + j(b - a)/n$. `mesh_cheby1.m` produces the zeros of T_{n+1} mapped to $[a, b]$, and `mesh_cheby2.m` produces the extrema of T_n mapped to $[a, b]$; both are returned in increasing order. `leja_order.m` reorders a given node set into Leja order by the greedy construction described in Berrut & Trefethen (2004, Section 3): at each step the next node is chosen to maximize $\prod_{k<j} |x_j - x_k|$ over the remaining candidates, starting from the node of largest absolute value.

4.2 Product Evaluation for Validation

`product_eval.m` evaluates $f(x) = \alpha \prod_{k=1}^{d}(x - \rho_k)$ in double precision using an incremental product, and serves as a high-accuracy reference wherever the test function is available in explicit product form.

4.3 Barycentric Form 1 Diagnostics

`bary1_coeffs.m` computes the Form 1 barycentric weights $\gamma_i = 1/\prod_{j \neq i}(x_i - x_j)$ in double precision. `bary1_eval_dp.m` evaluates the interpolant using these weights, also in double precision, and additionally returns the absolute numerator $\sum_i |\gamma_i f_i/(x - x_i)| \cdot |\omega(x)|$ and $|p_n(x)|$ at each evaluation point. These outputs feed directly into `cond_kappa` for the condition number calculation, and serve as the "exact" reference in tasks where no product form is available (Tasks 4 and 5).

4.4 Barycentric Form 2 Evaluation

`bary2_coeffs.m` computes the Form 2 barycentric weights using $O(n)$ closed-form expressions from Berrut & Trefethen (2004, equations 5.1, 5.3, 5.4) for uniform, Chebyshev type 1, and Chebyshev type 2 nodes respectively. These exploit the structure of each node distribution and avoid the $O(n^2)$ product formula needed for a general node set. `bary2_eval.m` evaluates the interpolant in single precision via the ratio formula, handling the case where an evaluation point coincides with a node exactly to avoid division by zero.

4.5 Newton Form: Coefficients and Evaluation

`newton_coeffs.m` computes the divided difference coefficients $\{a_0, \ldots, a_n\}$ in single precision using the standard in-place recurrence. `newton_eval.m` evaluates the resulting Newton form by accumulating from the highest-order term downward:

$$p_n(x) = a_0 + (x - x_0)\big(a_1 + (x - x_1)\big(a_2 + \cdots\big)\big).$$

The three node orderings: increasing, decreasing, and Leja are passed to these routines to study the effect of ordering on numerical accuracy.

4.6 Lebesgue Constant and Conditioning Quantities

`cond_kappa.m` computes the pointwise quantities $\mathrm{cond}(x, n, f)$ and $\mathrm{cond}(x, n, 1)$ defined in Section 3.5 over a dense evaluation grid, using the outputs of `bary1_eval_dp` for the numerator and denominator. `lebesgue_lambda.m` takes $\mathrm{cond}(x, n, 1)$ as input and returns $\Lambda_n = \max_x \kappa(x, n, 1)$. `hilbert_H.m` takes $\kappa(x, n, f)$ as input and returns $H_n = \max_x \kappa(x, n, f)$. These three routines together form the conditioning method used throughout the task drivers.

4.7 Hilbert Matrix Diagnostic

`hilbert_H.m` computes the quantity H_n reported in the conditioning tables as an additional sensitivity diagnostic alongside Λ_n and κ.

5 Driver Programs and Experiments

A separate driver was written for each task. Each driver calls the routines from Section 4 with the appropriate parameters and generates the tables and figures in Section 6. All figures are collected in Appendix A.

5.1 Task 1: Code Validation

The Task 1 driver verifies that all subroutines are working correctly using the polynomial $f_1(x) = (x-2)^9$ on $[0.5, 3.5]$ at $n = 9$. Because the exact values of f_1 are available from the product $(x-2)^9$, errors in any evaluation routine can be measured precisely. The driver checks mesh node coordinates, validates `product_eval` against a direct MATLAB computation, evaluates f_1 using all four forms on both meshes, and computes Λ_n and H_n for the uniform mesh.

5.2 Task 2: Interpolation of $f_1(x) = (x-2)^9$

The Task 2 driver repeats the interpolation of f_1 at $n = 9$ with a full conditioning and stability analysis across all three mesh types. It computes Λ_n, H_n, and the pointwise condition numbers on each mesh, evaluates f_1 in single precision using all four forms, and produces pointwise error plots with the Higham stability bound overlaid for each mesh. Figure 6 isolates the comparison between the barycentric Form 2 error and the stability bound on the uniform mesh.

5.3 Task 3: Interpolation of $f_2(x) = \prod_{i=1}^{d}(x-i)$

The Task 3 driver studies $f_2(x) = \prod_{i=1}^{d}(x-i)$ at degrees $d \in \{10, 21, 25\}$ on $[0, d+1]$. For each degree and mesh type it records conditioning quantities, errors across all four evaluation forms, and the results of overdetermined tests in which more nodes than required are used.

5.4 Task 4: Interpolation of $f_3(x) = \ell_n(x)$

Task 4 interpolates the n-th Lagrange basis function $f_3(x) = \ell_n(x)$ at degrees $n \in \{21, 25, 29\}$ on $[-1, 1]$. Because f_3 is defined by the mesh itself, double-precision **barycentric Form 1** evaluation

serves as the reference. The driver runs the same conditioning and accuracy sweep as Tasks 2–3, including overdetermined tests.

5.5 Task 5: Interpolation of $f_4(x) = 1/(1 + 25x^2)$

Task 5 studies $f_4(x) = 1/(1 + 25x^2)$ on $[-1, 1]$. Unlike the previous tasks, f_4 is not a polynomial, so the error reflects genuine approximation quality. The driver runs a conditioning sweep over $n \in \{5, 10, 15, 20, 25, 29, 35, 40, 50\}$, produces error plots at $n = 29$, and tracks $\|f_4 - p_n\|_\infty$ as n increases to study convergence behavior on each mesh type.

6 Results and Discussion

6.1 Task 1: Code Validation

Table 1 shows the node coordinates for each mesh type. The **uniform** and **Chebyshev type 2** meshes include the endpoints $a = 0.5$ and $b = 3.5$ exactly; **Chebyshev type 1** nodes cluster near the endpoints but do not include them. The Leja ordering begins at $x = 3.5$ and selects nodes greedily outward.

Table 2 shows that `product_eval` matches the exact double-precision value of f_1 to within 7.1×10^{-15}, confirming it introduces no meaningful error.

Form	Uniform	Cheby1
Barycentric Form 2	3.10×10^{-5}	2.34×10^{-5}
Newton increasing	1.70×10^{-4}	6.97×10^{-4}
Newton decreasing	2.43×10^{-4}	7.44×10^{-4}
Newton Leja	2.72×10^{-5}	2.34×10^{-5}

Table 1: Task 1, Table 3: Maximum interpolation error for f_1, $n = 9$.

Barycentric Form 2 and **Newton Leja** produce the smallest errors on both meshes, while **Newton increasing** and **decreasing** are larger by roughly an order of magnitude. This suggests that node ordering has an effect on the **Newton form** even at degree 9.

Table 4 reports $\Lambda_9 = 1.78 \times 10^1$ for the **uniform mesh**. The corresponding $H_n = 5.2 \times 10^{21}$ is large, but this is a consequence of $f_1 = (x - 2)^9$ having very large magnitude away from its root at $x = 2$ while being nearly zero nearby. H_n ends up large by construction of the function, not because of any instability in the algorithm.

6.2 Task 2: Interpolation of $f_1(x) = (x-2)^9$

Mesh	Λ_n	H_n
Uniform	1.78×10^1	2.69×10^{24}
Cheby1	2.43×10^0	7.29×10^{25}
Cheby2	2.36×10^0	2.71×10^{26}

Table 2: Task 2, Table 1: Conditioning summary for f_1, $n = 9$.

The **Chebyshev Lebesgue constants** are nearly identical and roughly seven times smaller than the uniform value. The large H_n values on all meshes again reflect the magnitude structure of f_1, not algorithmic behavior.

Form	Uniform	Cheby1	Cheby2
Barycentric Form 2	3.65×10^{-5}	2.75×10^{-5}	2.97×10^{-5}
Newton increasing	2.16×10^{-4}	7.14×10^{-4}	4.87×10^{-4}
Newton decreasing	1.61×10^{-4}	6.63×10^{-4}	7.54×10^{-4}
Newton Leja	2.75×10^{-5}	2.75×10^{-5}	2.75×10^{-5}

Table 3: Task 2, Table 2: Maximum interpolation error for f_1, all forms and meshes.

Barycentric Form 2 achieves comparable accuracy on all three meshes, and **Newton Leja** matches it exactly. **Newton increasing** and **decreasing** are consistently worse by roughly an order of magnitude regardless of mesh. At this degree the evaluation form matters more than the mesh distribution.

Figures 3–5 confirm that observed errors stay below the Higham stability bound on all meshes. Figure 6 shows this directly for the uniform mesh, where the **barycentric Form 2** error tracks the bound closely.

6.3 Task 3: Interpolation of $f_2(x) = \prod_{i=1}^{d}(x-i)$

d	Mesh	Λ_n	H_n
10	Uniform	2.99×10^1	1.38×10^3
10	Cheby1	2.49×10^0	4.23×10^4
21	Uniform	2.06×10^4	7.98×10^2
21	Cheby1	2.93×10^0	1.12×10^7
25	Uniform	2.61×10^5	9.94×10^2
25	Cheby1	3.04×10^0	1.59×10^8

Table 4: Task 3, Table 1: Conditioning summary for f_2, selected degrees and meshes.

The **uniform mesh** Λ_n grows by roughly four orders of magnitude from $d = 10$ to $d = 25$; the **Chebyshev** value barely moves. This is the exponential versus logarithmic growth predicted by theory.

d	Bary2 Uniform	Bary2 Cheby1
10	5.54×10^0	4.54×10^0
21	9.46×10^{15}	1.26×10^{14}
25	5.47×10^{22}	3.86×10^{19}

Table 5: Task 3, Table 2: Maximum barycentric Form 2 error for f_2 across degrees.

At $d = 10$ the errors are of order one, already reflecting the large magnitude of f_2 rather than algorithmic failure. By $d = 25$ the errors are enormous, and notably they are far larger than the Higham bound predicts: at $d = 25$ on the uniform mesh, $H_n \cdot \Lambda_n \cdot u \approx 10^3 \times 2.6 \times 10^5 \times 6 \times 10^{-8} \approx 10^1$, yet the observed error is 5.47×10^{22}. The errors are therefore not simply rounding errors amplified by Λ_n. They reflect the intrinsic ill-conditioning of the polynomial f_2 itself. This is precisely the phenomenon documented by Wilkinson in "The Perfidious Polynomial": even though the roots of f_2 are well-separated integers, the polynomial's representation in any standard basis is catastrophically sensitive to small perturbations, because the coefficient magnitudes grow enormously with degree, making floating point evaluation near the roots unreliable regardless of the algorithm used. The Chebyshev errors are several orders smaller, confirming that Λ_n still plays a role, but neither mesh escapes the fundamental ill-conditioning of f_2 at these degrees.

Table 3 shows the overdetermined tests. At 31 **uniform nodes** for a degree-25 polynomial the error grows to 2.9×10^5, while at 16 **Chebyshev nodes** for a degree-10 polynomial the **barycentric Form 2** error stays near 4.0, essentially unchanged from the exact-degree case. Overdetermination is harmful on the uniform mesh and neutral at best on Chebyshev.

6.4 Task 4: Interpolation of $f_3(x) = \ell_n(x)$

The Lagrange basis function $f_3(x) = \ell_n(x)$ satisfies $\|f_3\|_\infty = 1$ by construction, which means $H_n = 1$ identically across all meshes and degrees. This makes Task 4 the cleanest test of the Higham bound: with $H_n = 1$, the stability estimate reduces to $|\hat{p}_n(x) - p_n(x)| \lesssim \Lambda_n u$, and any error growth is attributable directly to Λ_n with no contribution from the function magnitude.

n	Bary2 Uniform	Bary2 Cheby1
21	1.17×10^{-4}	3.41×10^{-6}
25	2.75×10^{-3}	9.44×10^{-7}
29	3.73×10^{-2}	7.13×10^{-6}

Table 6: Task 4, Table 2: Maximum barycentric Form 2 error for f_3 across degrees.

The **uniform mesh** errors grow by roughly one order of magnitude every five degrees, tracking the growth of Λ_n from 2.06×10^4 at $n = 21$ to 3.45×10^6 at $n = 29$. The **Chebyshev type 1**

errors stay near 10^{-6} throughout, consistent with $\Lambda_n \approx 3$.

This is the same experiment Higham (2004) used to distinguish Form 1 and Form 2. He worked in double precision ($u \approx 10^{-16}$) and found large relative errors near the endpoints on uniform nodes. Our single-precision experiments ($u \approx 6 \times 10^{-8}$) yield a maximum error of 3.73×10^{-2} at $n = 29$ on the uniform mesh. Since the two precisions differ by a factor of roughly 6×10^8, single-precision errors on the same mesh should be approximately that many times larger than Higham's, which is consistent with what we observe. The qualitative picture is identical: error grows with Λ_n on uniform meshes and remains controlled on Chebyshev meshes.

Table 3 shows the overdetermined tests. At 36 **Chebyshev nodes**, **barycentric Form 2** achieves 1.8×10^{-7}, near the single-precision floor, while **Newton increasing** on the same mesh gives 5.5×10^1 — a gap of eight orders of magnitude. At 46 **uniform nodes**, the **barycentric Form 2** error is ∞. This is the extreme case described by Higham's Corollary 4.2: Λ_n has grown large enough that the denominator of the barycentric formula suffers total cancellation in single precision, evaluating to zero and producing NaN or Inf. The contrast with the Chebyshev result is stark, and is a direct consequence of the exponential growth of Λ_n on the uniform mesh.

6.5 Task 5: Runge's Function and Convergence

Form	Uniform	Cheby1	Cheby2
Barycentric Form 2	3.59×10^2	5.15×10^{-3}	6.16×10^{-3}
Newton increasing	3.34×10^2	5.65×10^0	2.00×10^0
Newton decreasing	3.34×10^2	5.65×10^0	2.00×10^0
Newton Leja	3.34×10^2	5.15×10^{-3}	6.16×10^{-3}

Table 7: Task 5, Table 2: Maximum interpolation error for f_4 at $n = 29$.

Unlike Tasks 2–4, f_4 is not a polynomial, so the approximation error reflects genuine approximation quality rather than representation error. Because $\|f_4\|_\infty \leq 1$, the H_n values are modest throughout, which means Task 5 provides the cleanest view of the Λ_n effect among all the tasks: any error growth is attributable directly to the growth of the Lebesgue constant rather than to the magnitude of the function.

The Λ_n sweep confirms the by-now familiar pattern: on **uniform meshes** Λ_n grows from 3.1 at $n = 5$ to 3.6×10^{12} at $n = 50$; on **Chebyshev type 1** it grows from 2.1 to 3.5 over the same range. At $n = 29$, all forms on the uniform mesh produce errors near 3.4×10^2, exceeding the maximum value of f_4 itself. The Runge phenomenon in full. On **Chebyshev meshes**, **barycentric Form 2** and **Newton Leja** achieve errors near 5×10^{-3}, while **Newton increasing** and **decreasing** produce errors of order 1-6. **Leja** ordering recovers **barycentric**-level accuracy on the well-conditioned mesh; a poor ordering wastes the advantage of **Chebyshev nodes** entirely.

The convergence sweep confirms the theoretical picture. On uniform meshes the error grows without bound. On **Chebyshev meshes** it decreases steadily until leveling off at the single-precision floor $u \approx 6 \times 10^{-8}$.

Target $\|f_4 - p_n\|_\infty$	Approximate n (Cheby1)
10^{-2}	$n \approx 10$
10^{-4}	$n \approx 20$–25
10^{-6}	$n \approx 35$
10^{-8}	$n \approx 50$

Table 8: Task 5: Approximate degree required to reach target accuracy on Cheby1 mesh.

These results also clarify the apparent tension between the two assigned papers. Berrut & Trefethen (2004) show that **barycentric interpolation** converges well on **Chebyshev meshes**; Higham (2004) warns that **barycentric Form 2** can fail when Λ_n is large. Table 2 exhibits both simultaneously: on the **uniform mesh** the error is 3.59×10^2; on **Chebyshev type 1** it is 5.15×10^{-3}. There is no contradiction; Berrut & Trefethen's convergence holds precisely because the **Chebyshev mesh** keeps Λ_n small, and Higham's instability analysis applies on meshes where it does not. Both papers describe the same underlying bound, approached from different directions.

7 Conclusions

The experiments confirm the theoretical predictions of Higham (2004) and Berrut & Trefethen (2004) across five tasks. The Lebesgue constant Λ_n is the central quantity governing numerical behavior, and its growth is controlled almost entirely by the mesh distribution.

- **Mesh distribution is the dominant factor.** Λ_n grows exponentially on **uniform meshes** and logarithmically on **Chebyshev meshes**. By $n = 29$, the uniform constant exceeds 3×10^6 while the **Chebyshev type 1** constant remains near 3. This drives the Runge phenomenon in Task 5 and the error growth in Tasks 3 and 4.

- **Barycentric Form 2 is the most reliable evaluation form.** It produced the smallest errors across all tasks and mesh types and tracked the Higham stability bound closely. Unlike the **Newton form**, its accuracy is independent of node ordering.

- **Node ordering critically affects the Newton form.** Increasing and decreasing orderings produced errors one to three orders of magnitude larger than **barycentric Form 2** on the same mesh in many experiments. **Leja** ordering recovered **barycentric**-level accuracy in nearly every case.

- **The Higham bound predicts error growth accurately — when applicable.** In Task 4, with $H_n = 1$ by construction, the **uniform mesh** error tracks $\Lambda_n \cdot u$ directly. In Task 3, the observed errors at $d = 25$ far exceed the bound, which is itself informative: the excess indicates that the dominant errors come from the intrinsic ill-conditioning of the polynomial (the Wilkinson phenomenon), not from rounding in the evaluation algorithm.

- **Overdetermination is not beneficial.** Using more nodes than the degree requires did not improve accuracy and in several cases caused substantial error growth.
- **Single-precision arithmetic sets a hard floor.** In Task 5, **Chebyshev mesh** errors stopped improving near $u \approx 6 \times 10^{-8}$ regardless of degree.

A well-conditioned mesh and a stable evaluation form are not optional refinements; they are necessary conditions for polynomial interpolation to be reliable in finite precision arithmetic.

---

8 Disclaimer

Claude was used this time to help draft and organize parts of this report and to help with formatting. I also used it to help design and debug parts of the driver code. I have reviewed all of the code and added comments throughout to ensure I understood what was going on.

A Appendix: Figures

All figures below were generated by the task drivers and are grouped by task. Multiple figures are arranged side by side to reduce page length.

Task 1: Code Validation

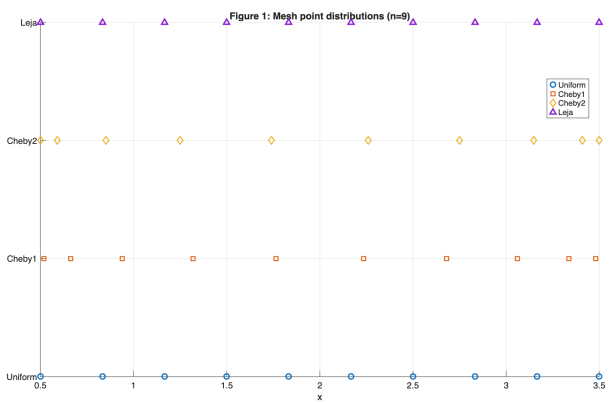

Figure 1: Task 1, Figure 1: Mesh point distributions, $n = 9$.

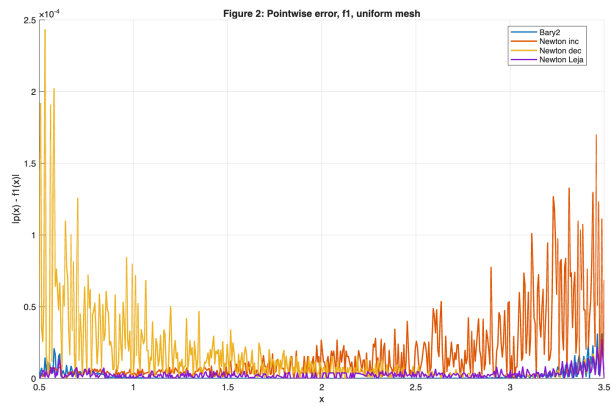

Figure 2: Task 1, Figure 2: Pointwise error, all forms, uniform mesh.

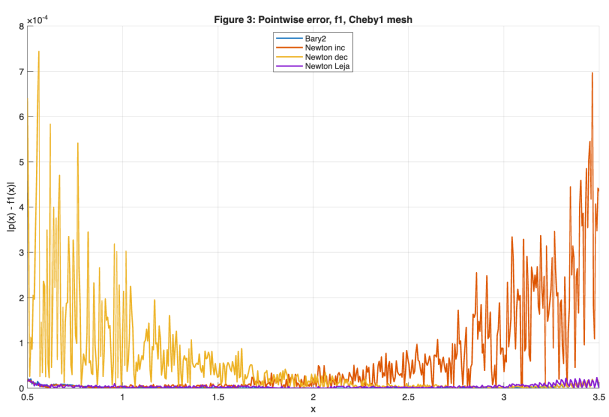

Figure 3: Task 1, Figure 3: Pointwise error, all forms, Cheby1 mesh.

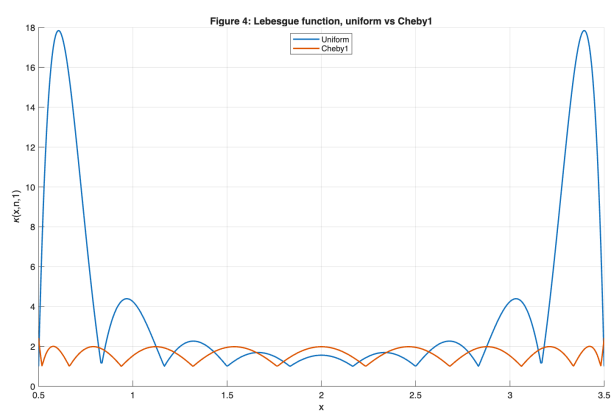

Figure 4: Task 1, Figure 4: Lebesgue function, uniform vs. Cheby1.

Task 2: Interpolation of $f_1(x) = (x-2)^9$

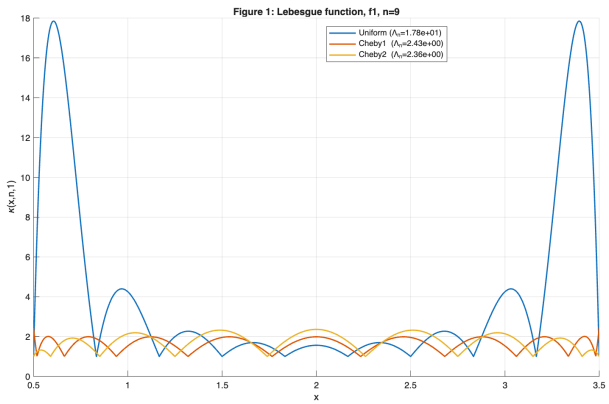

Figure 5: Task 2, Figure 1: Lebesgue function, all meshes.

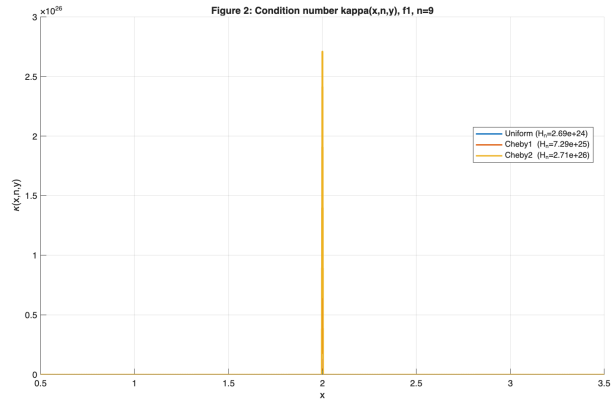

Figure 6: Task 2, Figure 2: Condition number $\kappa(x, n, y)$, all meshes.

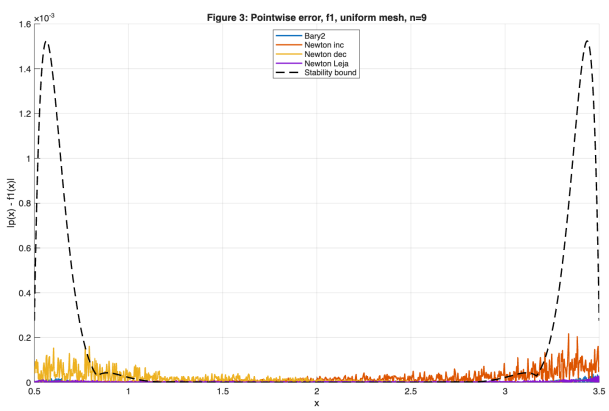

Figure 7: Task 2, Figure 3: Pointwise error, all forms, uniform mesh.

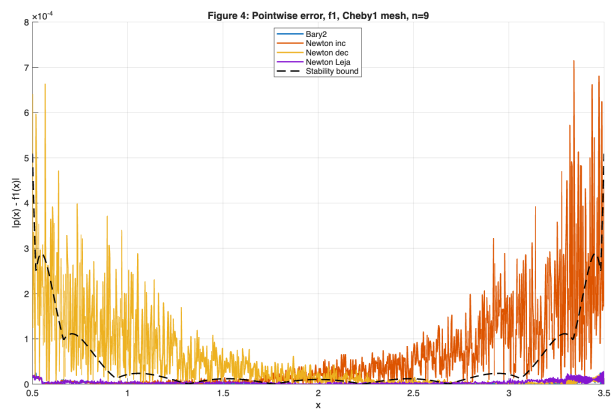

Figure 8: Task 2, Figure 4: Pointwise error, all forms, Cheby1 mesh.

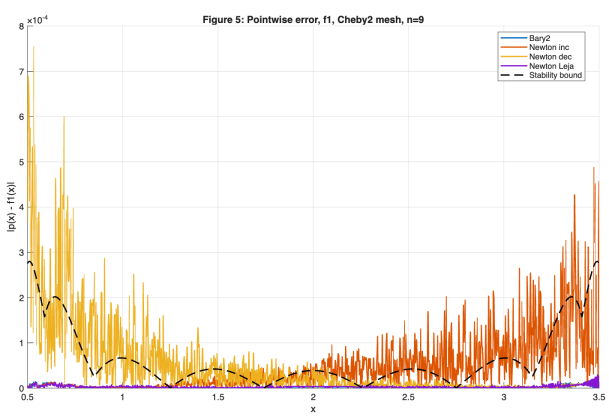

Figure 9: Task 2, Figure 5: Pointwise error, all forms, Cheby2 mesh.

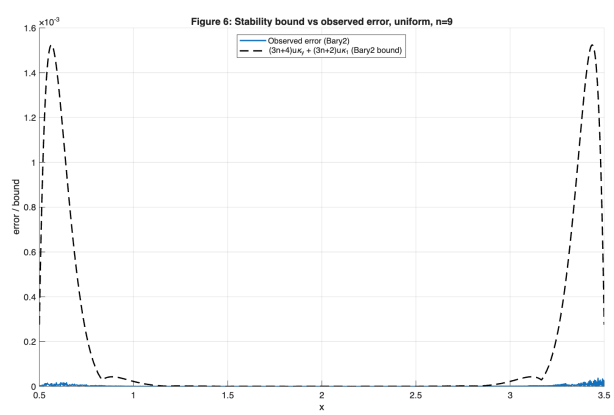

Figure 10: Task 2, Figure 6: Stability bound vs. observed error, uniform mesh.

Task 3: Interpolation of $f_2(x) = \prod_{i=1}^{d}(x-i)$

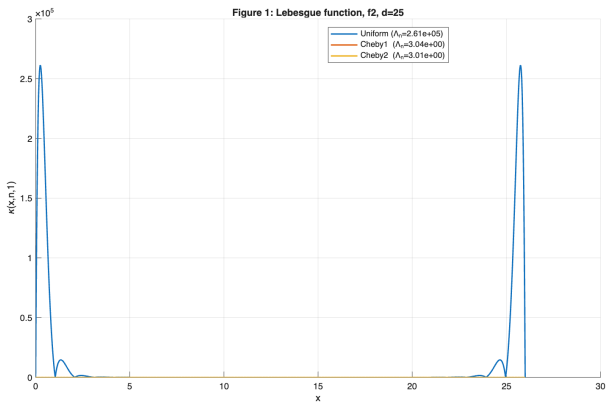

Figure 11: Task 3, Figure 1: Lebesgue function, all meshes, $d = 25$.

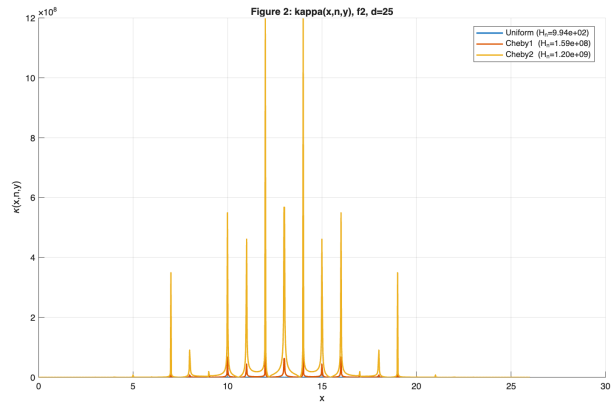

Figure 12: Task 3, Figure 2: Condition number $\kappa(x, n, y)$, $d = 25$.

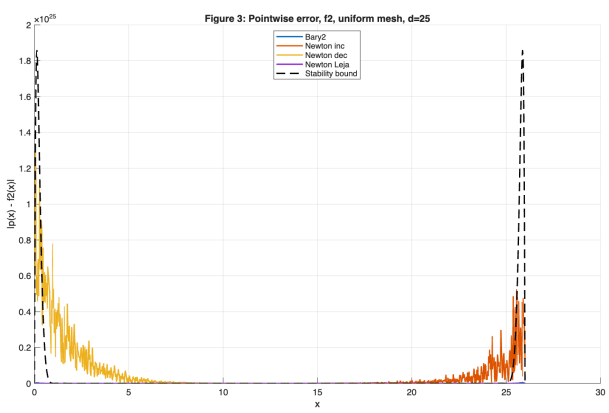

Figure 13: Task 3, Figure 3: Pointwise error, all forms, uniform mesh, $d = 25$.

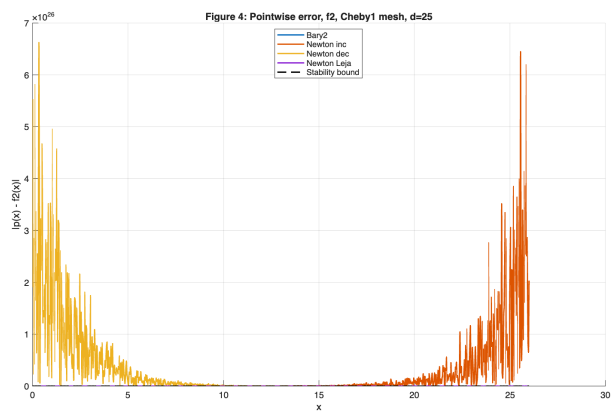

Figure 14: Task 3, Figure 4: Pointwise error, all forms, Cheby1 mesh, $d = 25$.

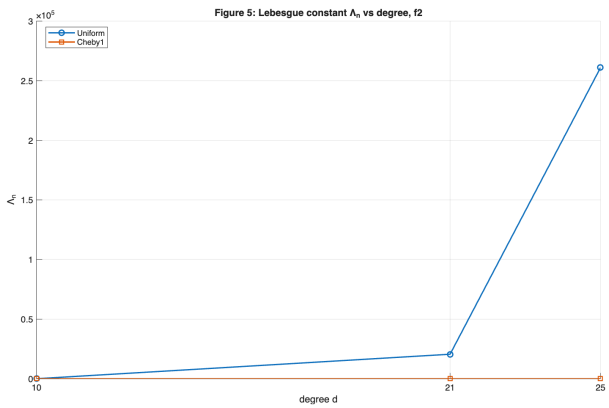

Figure 15: Task 3, Figure 5: Λ_n vs. degree, uniform and Cheby1.

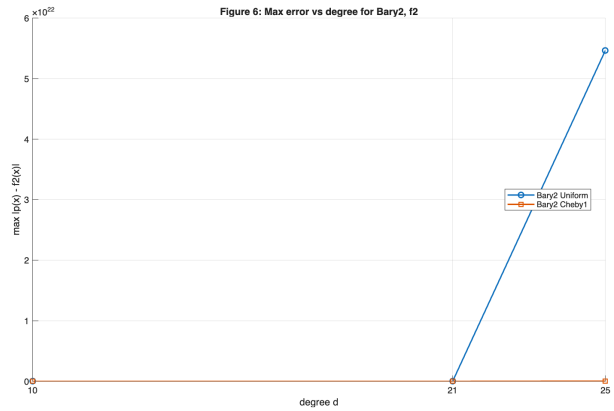

Figure 16: Task 3, Figure 6: Max error vs. degree, Bary2 on all meshes.

Task 4: Interpolation of $f_3(x) = \ell_n(x)$

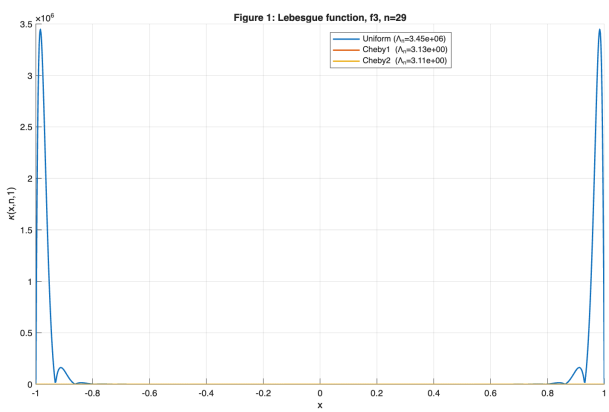

Figure 17: Task 4, Figure 1: Lebesgue function, all meshes, $n = 29$.

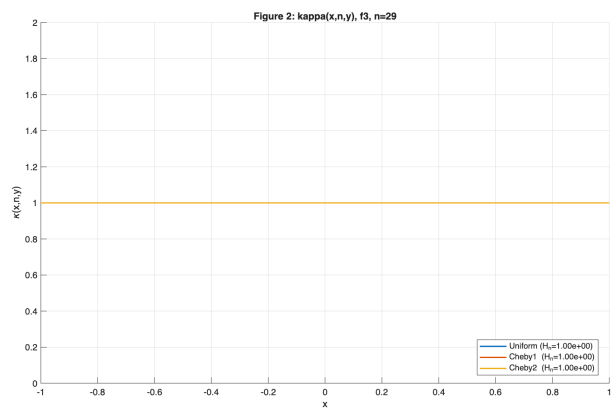

Figure 18: Task 4, Figure 2: Condition number $\kappa(x, n, y)$, $n = 29$.

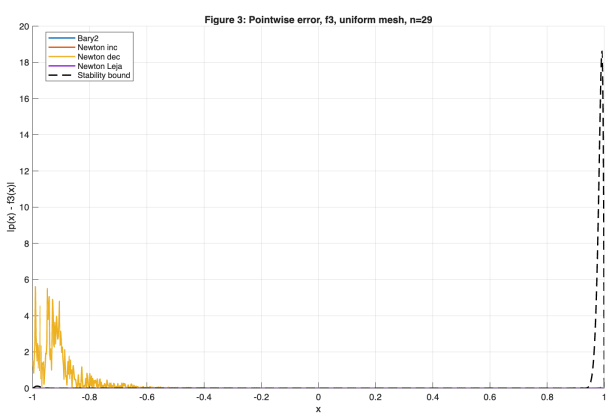

Figure 19: Task 4, Figure 3: Pointwise error, all forms, uniform mesh, $n = 29$.

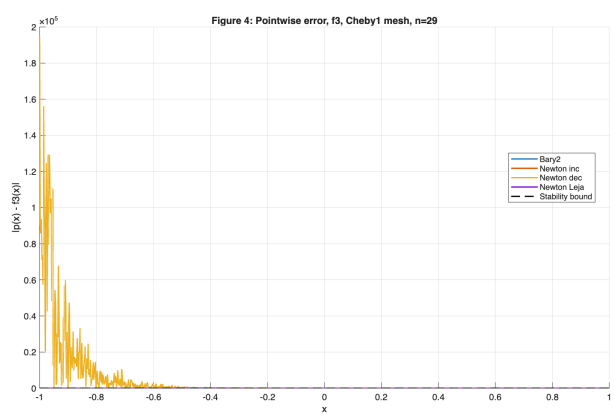

Figure 20: Task 4, Figure 4: Pointwise error, all forms, Cheby1 mesh, $n = 29$.

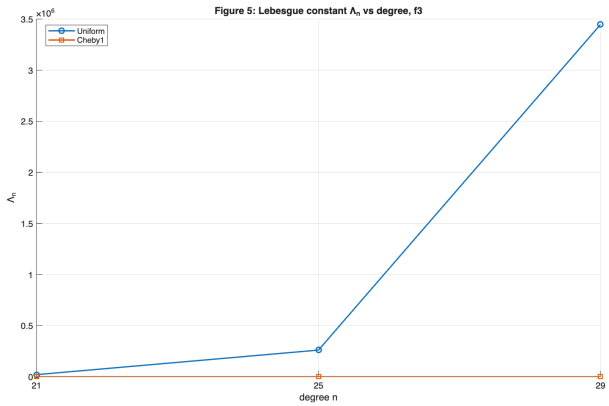

Figure 21: Task 4, Figure 5: Λ_n vs. degree, uniform and Cheby1.

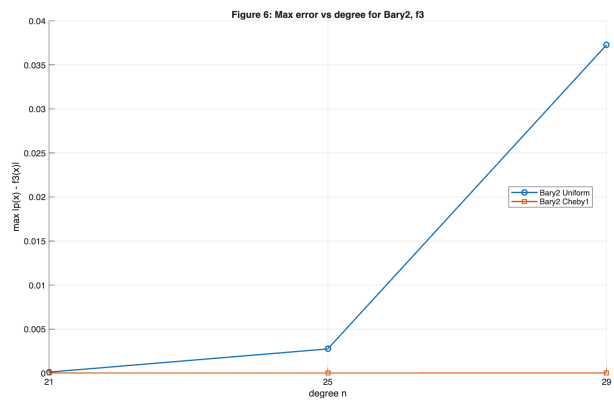

Figure 22: Task 4, Figure 6: Max error vs. degree, Bary2 on all meshes.

Task 5: Runge's Function $f_4(x) = 1/(1 + 25x^2)$

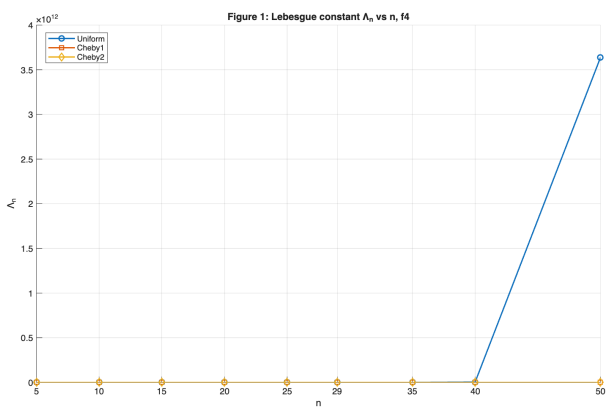

Figure 23: Task 5, Figure 1: Λ_n vs. n, all meshes.

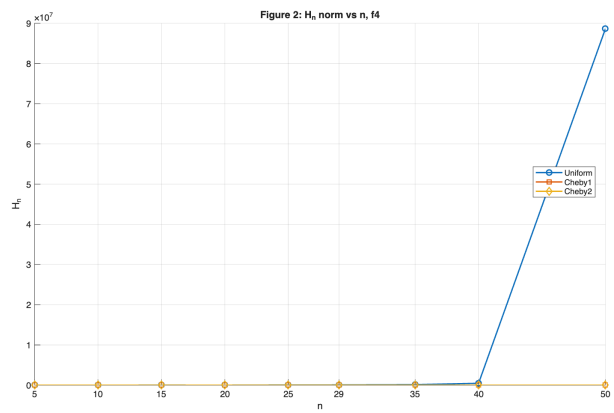

Figure 24: Task 5, Figure 2: H_n vs. n, all meshes.

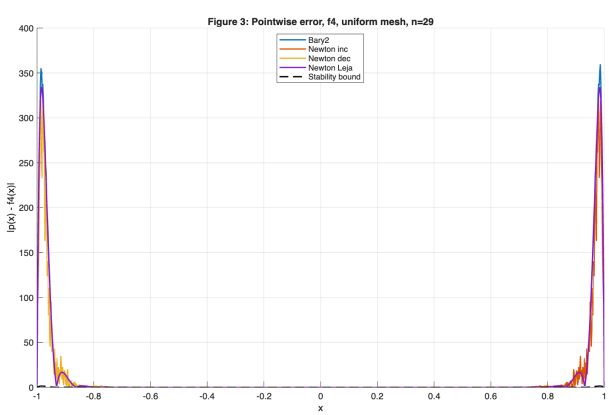

Figure 25: Task 5, Figure 3: Pointwise error, all forms, uniform mesh, $n = 29$.

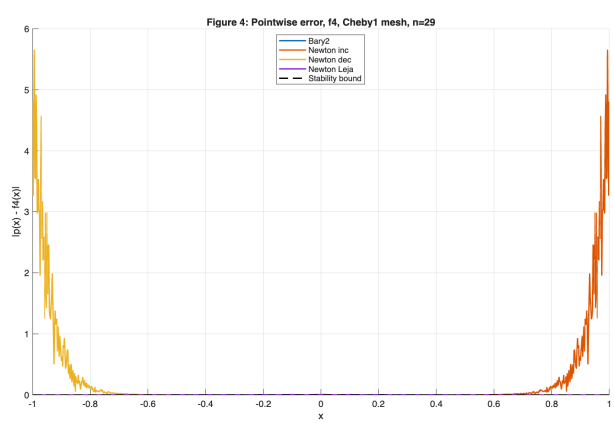

Figure 26: Task 5, Figure 4: Pointwise error, all forms, Cheby1 mesh, $n = 29$.

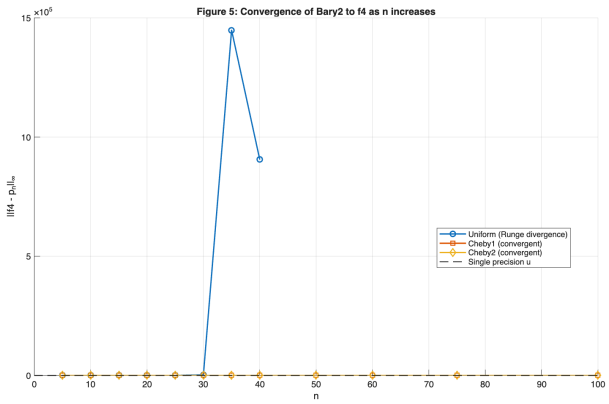

Figure 27: Task 5, Figure 5: $\|f_4 - p_n\|_\infty$ vs. n, all meshes.

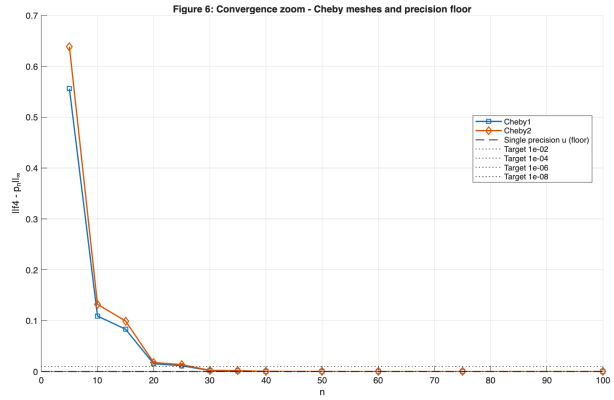

Figure 28: Task 5, Figure 6: Convergence zoom, Cheby meshes with precision floor.